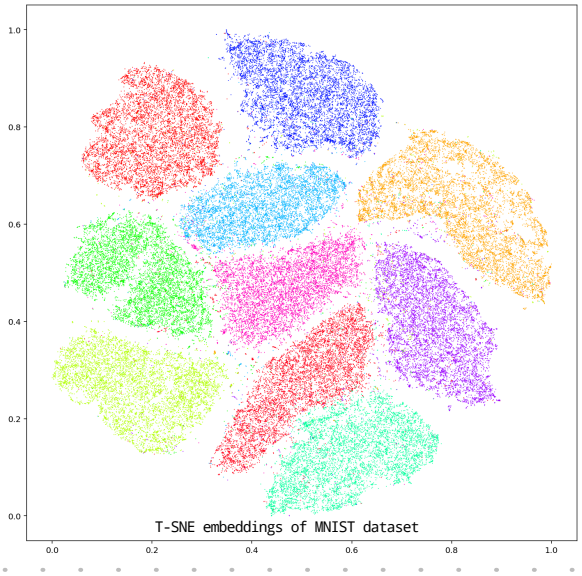
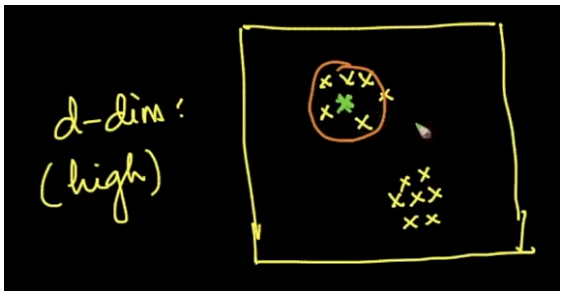
# t-distributed stochastic neighbor embedding

t-distributed stochastic neighbor embedding (t-SNE) is a statistical method for visualizing high-dimensional data by giving each datapoint a location in a two or three-dimensional map.

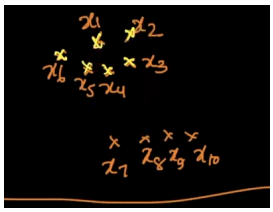
PCA preserves global structure
t-SNE preserves local structure



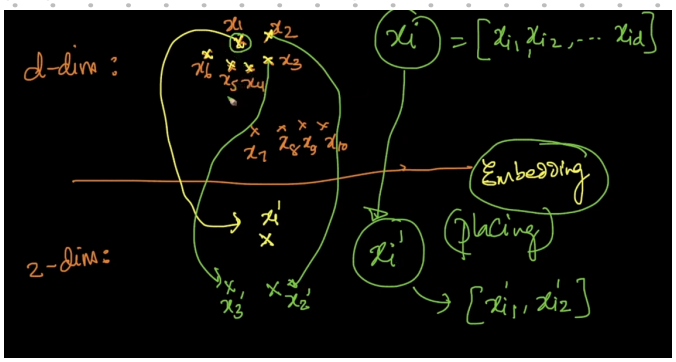




neighborhood of a point is all those points geometrically close to it.

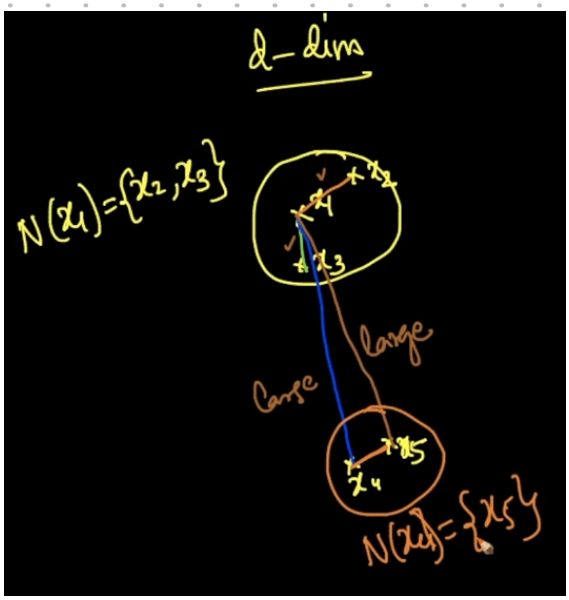
embedding:



say if we have data like this,
we take points from the high dim space one by one and place corresp. points in the low dim space



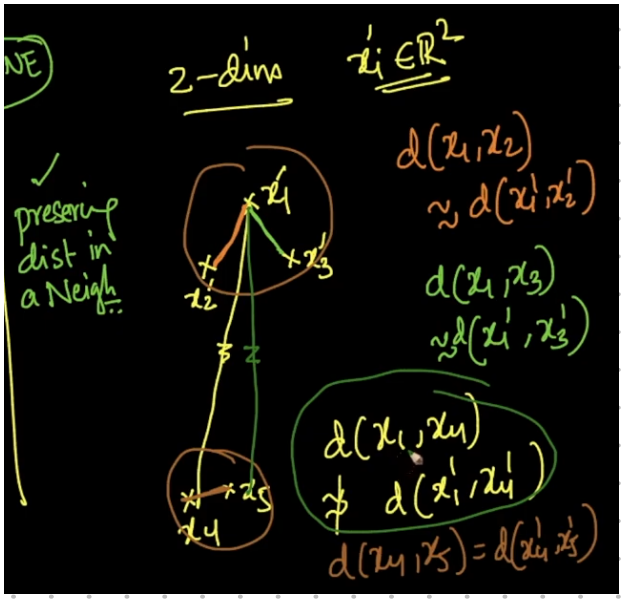
geometric intuition:



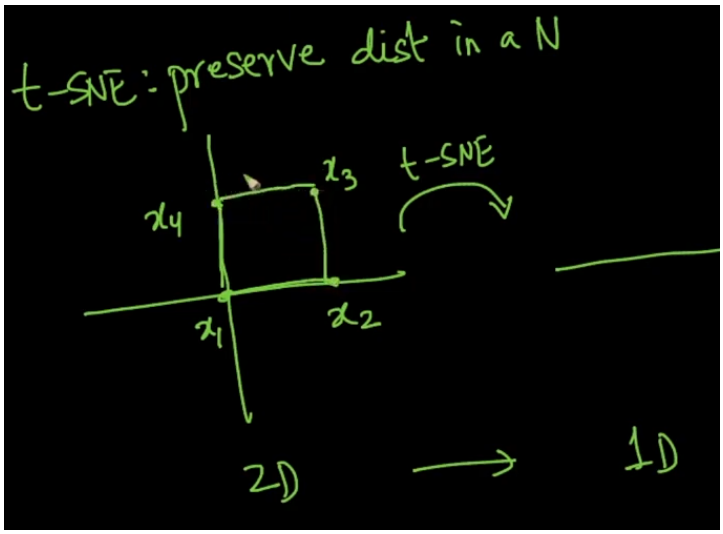

say we have data like this, then N(X1), the neighborhood of x1, has {X2, X3}
lly, N(X4) = {X5}

t-SNE would try to embed all xi's onto a lower dim space
distance betn points would be roughly equal
neighborhoods would be preserved

points not in a neighborhood may not share the same dist in the lower space




crowding problem:

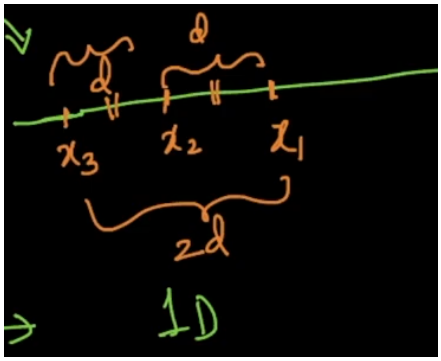



say we have 4 points,
they are corners of a square
they are at equal distances from adj points
say side of sq has distance d

(in d-dim space they could be corners of a hypercube)

in 1-d, we can't possibly try and make distances same

say our neighborhood is defined as points with distance d
so x2 will have x3 and x1 in neighborhood
x1 will have x2 and x4 in neighborhood
d(x1, x3) is sqrt(2)*d




x3 doesn't belong to neighborhood of x1, so preserving sqrt2 d is not necessary
but now if we try to place x4, we can't make it at d from both x3 and x1 as d at the same time,
even though both are in its neighborhood
so that's the neighborhood problem

so sometimes it is impossible to preserve all the distances in a neighborhood

How to use t-SNE

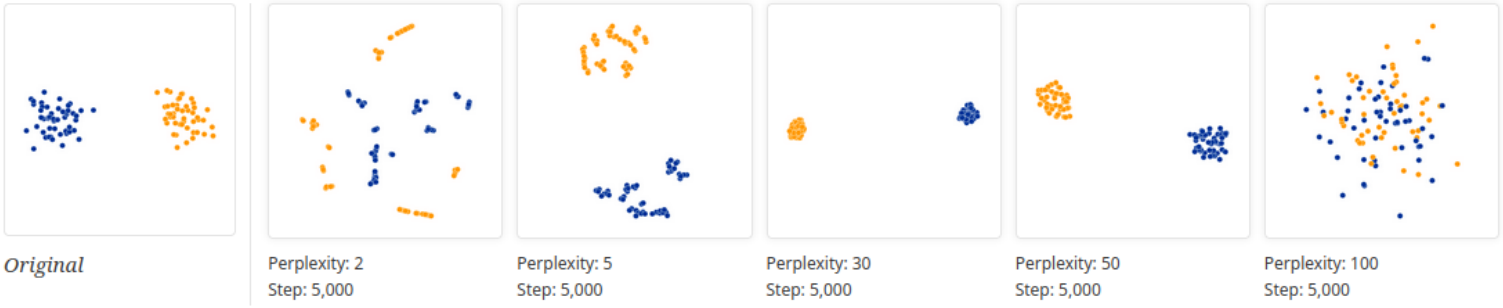
t-SNE is an iterative algorithm
it tries to improve the embedding in every iteration

parameters:

steps:
# of iterations (larger # of iterations helps stabilize the output)

perplexity:
# of neighbors to preserve
(like, say instead of preserving d distance points, we'll choose to preserve exactly 5 points)

examples:

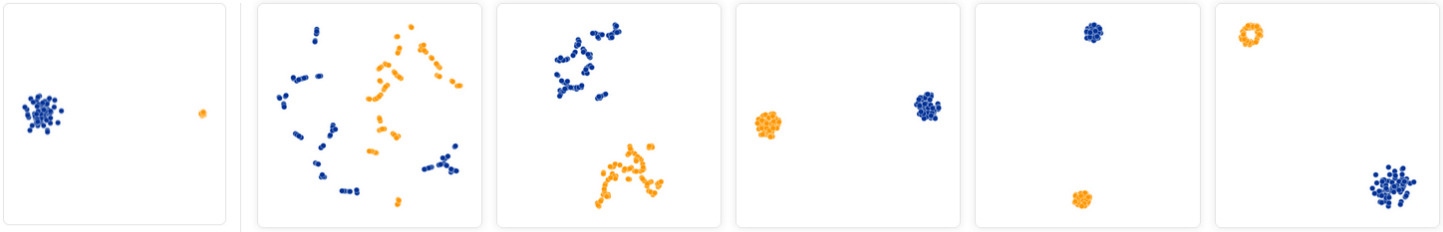



for the algorithm to operate properly,
the perplexity really should be smaller than the number of points.




The images above show five different runs at perplexity 30. The first four were stopped before stability. After 10, 20, 60, and 120 steps you can see layouts with seeming 1-dimensional and even pointlike images of the clusters. If you see a t-SNE plot with strange "pinched" shapes, chances are the process was stopped too early. Unfortunately, there's no fixed number of steps that yields a stable result. Different data sets can require different numbers of iterations to converge.

# Cluster sizes in a t-SNE plot mean nothing



*Original*

Perplexity: 2
Step: 5,000

Perplexity: 5
Step: 5,000

Perplexity: 30
Step: 5,000

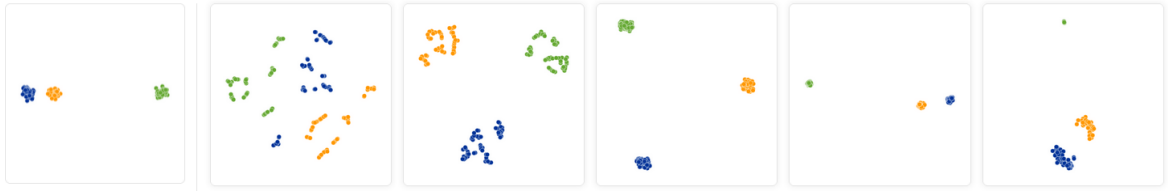
Perplexity: 50
Step: 5,000

Perplexity: 100
Step: 5,000

Surprisingly, the two clusters look about same size in the t-SNE plots. What's going on? The t-SNE algorithm adapts its notion of "distance" to regional density variations in the data set. As a result, it naturally expands dense clusters, and contracts sparse ones, evening out cluster sizes. To be clear, this is a different effect than the run-of-the-mill fact that any dimensionality reduction technique will distort distances. (After all, in this example all data was two-dimensional to begin with.) Rather, density equalization happens by design and is a predictable feature of t-SNE.

The bottom line, however, is that you cannot see relative sizes of clusters in a t-SNE plot.

# Distances between clusters might not mean anything



*Original*

Perplexity: 2
Step: 5,000

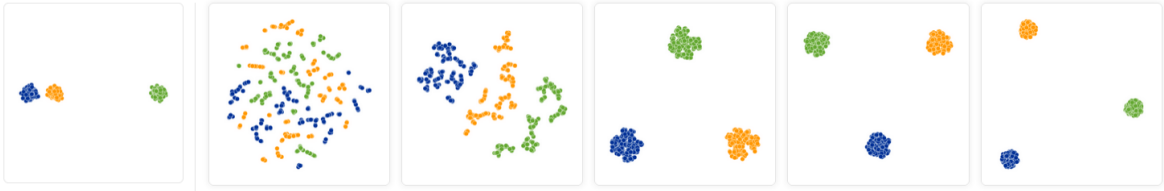
Perplexity: 5
Step: 5,000

Perplexity: 30
Step: 5,000

Perplexity: 50
Step: 5,000

Perplexity: 100
Step: 5,000

# global distances seem preserved here, but....



*Original*

Perplexity: 2
Step: 5,000

Perplexity: 5
Step: 5,000

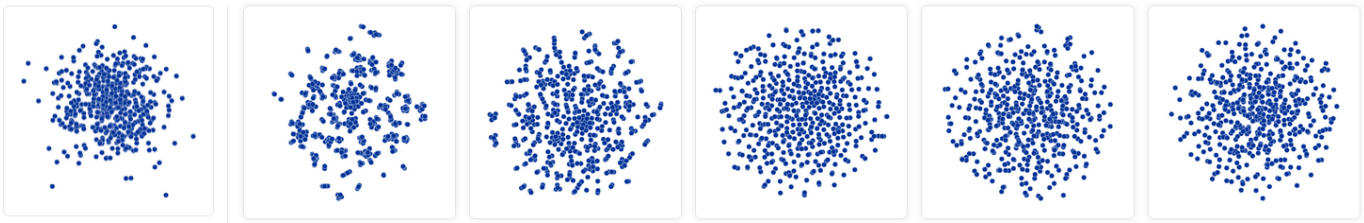
Perplexity: 30
Step: 5,000

Perplexity: 50
Step: 5,000

Perplexity: 100
Step: 5,000

# we've increased # of points from 50 each to 200 each, global distance is fucked

## Random noise doesn't always look random.

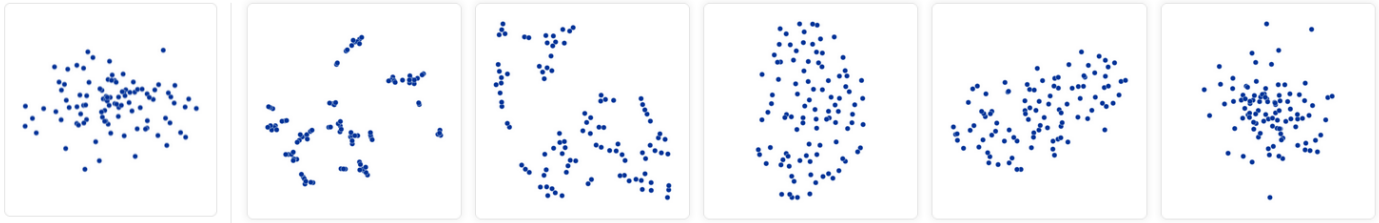


*Original* | Perplexity: 2 Step: 5,000 | Perplexity: 5 Step: 5,000 | Perplexity: 30 Step: 5,000 | Perplexity: 50 Step: 5,000 | Perplexity: 100 Step: 5,000

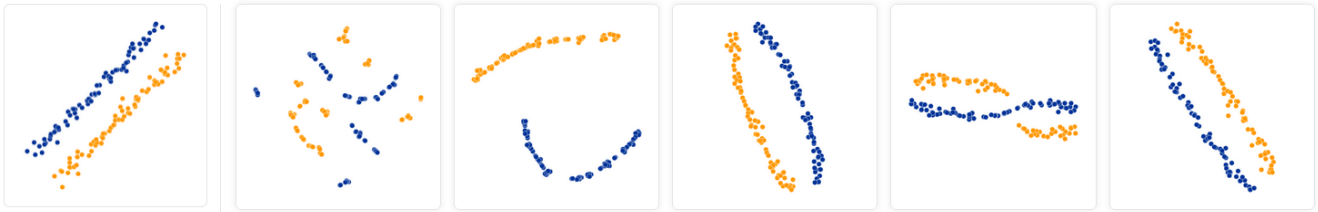
The plot with perplexity 2 seems to show dramatic clusters. If you were tuning perplexity to bring out structure in the data, you might think you'd hit the jackpot.

Of course, since we know the cloud of points was generated randomly, it has no statistically interesting clusters: those "clumps" aren't meaningful. If you look back at previous examples, low perplexity values often lead to this kind of distribution. Recognizing these clumps as random noise is an important part of reading t-SNE plots.

## You can see some shapes, sometimes



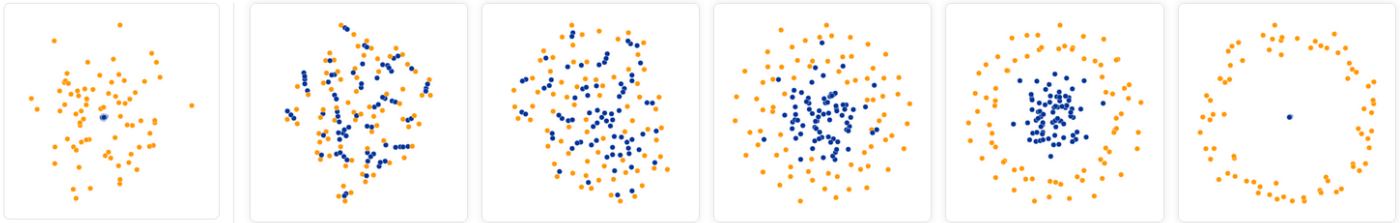
*Original* | Perplexity: 2 Step: 5,000 | Perplexity: 5 Step: 5,000 | Perplexity: 30 Step: 5,000 | Perplexity: 50 Step: 5,000 | Perplexity: 100 Step: 5,000



*Original* | Perplexity: 2 Step: 5,000 | Perplexity: 5 Step: 5,000 | Perplexity: 30 Step: 5,000 | Perplexity: 50 Step: 5,000 | Perplexity: 100 Step: 5,000

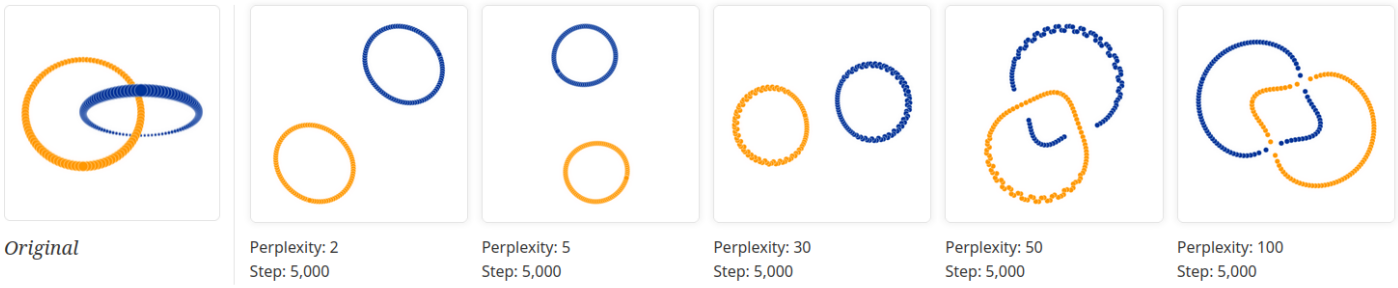
Even in the best cases, though, there's a subtle distortion: the lines are slightly curved outwards in the t-SNE diagram. The reason is that, as usual, t-SNE tends to expand denser regions of data. Since the middles of the clusters have less empty space around them than the ends, the algorithm magnifies them.

## For topology, you may need more than one plot



*Original* | Perplexity: 2 Step: 5,000 | Perplexity: 5 Step: 5,000 | Perplexity: 30 Step: 5,000 | Perplexity: 50 Step: 5,000 | Perplexity: 100 Step: 5,000

The perplexity 30 view shows the basic topology correctly, but again t-SNE greatly exaggerates the size of the smaller group of points. At perplexity 50, there's a new phenomenon: the outer group becomes a circle, as the plot tries to depict the fact that all its points are about the same distance from the inner group. If you looked at this image alone, it would be easy to misread these outer points as a one-dimensional structure.



*Original*

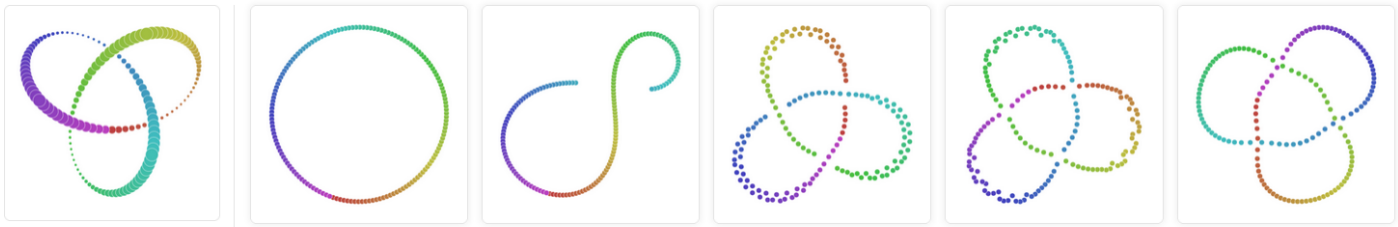
Perplexity: 2
Step: 5,000

Perplexity: 5
Step: 5,000

Perplexity: 30
Step: 5,000

Perplexity: 50
Step: 5,000

Perplexity: 100
Step: 5,000



*Original*

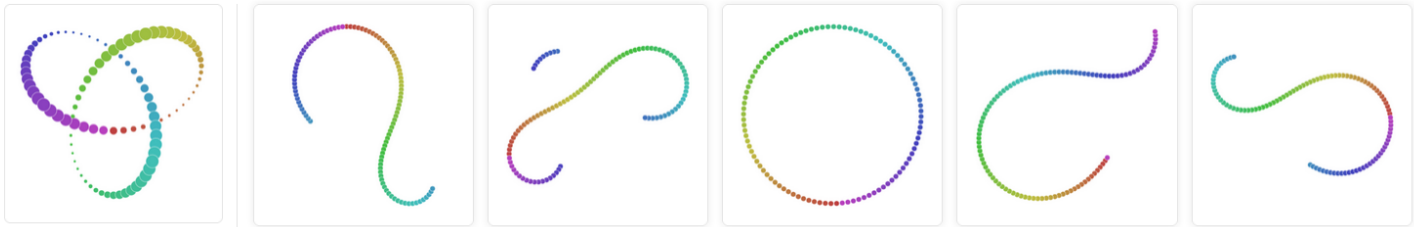
Perplexity: 2
Step: 5,000

Perplexity: 5
Step: 5,000

Perplexity: 30
Step: 5,000

Perplexity: 50
Step: 5,000

Perplexity: 100
Step: 5,000



*Original*

Perplexity: 2
Step: 5,000

Perplexity: 2
Step: 5,000

Perplexity: 2
Step: 5,000

Perplexity: 2
Step: 5,000

Perplexity: 2
Step: 5,000

## Conclusion

There's a reason that t-SNE has become so popular: it's incredibly flexible, and can often find structure where other dimensionality-reduction algorithms cannot. Unfortunately, that very flexibility makes it tricky to interpret. Out of sight from the user, the algorithm makes all sorts of adjustments that tidy up its visualizations. Don't let the hidden "magic" scare you away from the whole technique, though. The good news is that by studying how t-SNE behaves in simple cases, it's possible to develop an intuition for what's going on.

```
run iterations till shape stabilizes
have multiple runs
rerun with diff perplexities
NEVER run t-SNE once
```

# How t-SNE works

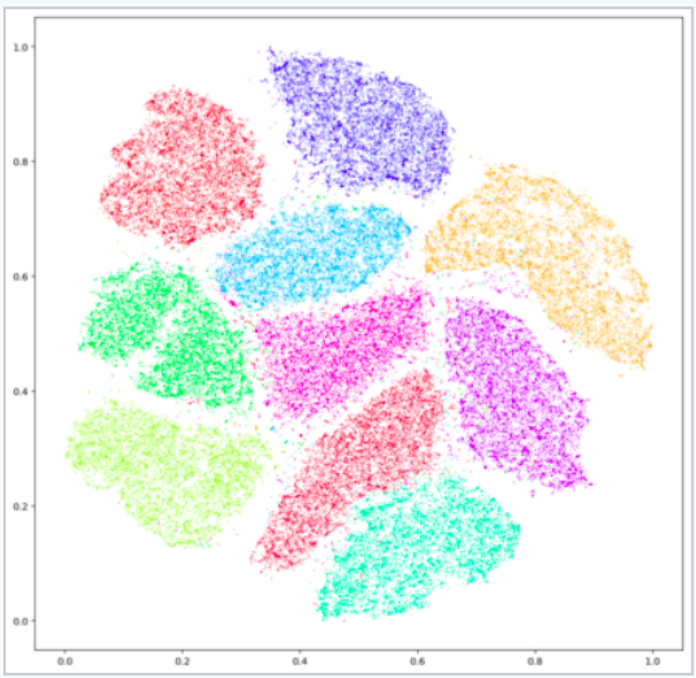
The t-SNE algorithm finds the similarity measure between pairs of instances in higher and lower dimensional space. After that, it tries to optimize two similarity measures. It does all of that in three steps.

1. t-SNE models a point being selected as a neighbor of another point in both higher and lower dimensions. It starts by calculating a pairwise similarity between all data points in the high-dimensional space using a Gaussian kernel. The points that are far apart have a lower probability of being picked than the points that are close together.
2. Then, the algorithm tries to map higher dimensional data points onto lower dimensional space while preserving the pairwise similarities.
3. It is achieved by minimizing the divergence between the probability distribution of the original high-dimensional and lower-dimensional. The algorithm uses gradient descent to minimize the divergence. The lower-dimensional embedding is optimized to a stable state.

t-SNE on MNIST

groups points with visual similarity together

class labels obv group together



T-SNE embeddings of MNIST dataset

```
from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, random_state=0)
```

✓ 0.0s

```
sd = standardized_data
lb = labels

tsne_data = tsne.fit_transform(sd)
```

✓ 1m 58.2s

```
sd.shape, lb.shape
```

✓ 0.0s

```
((42000, 784), (42000,))
```

```
tsne_data.shape
```

✓ 0.0s

```
(42000, 2)
```

```
tsne_data_l = np.vstack((tsne_data.T, lb)).T
tsne_df = pd.DataFrame(tsne_data_l, columns=["dim1", "dim2", "label"])
```

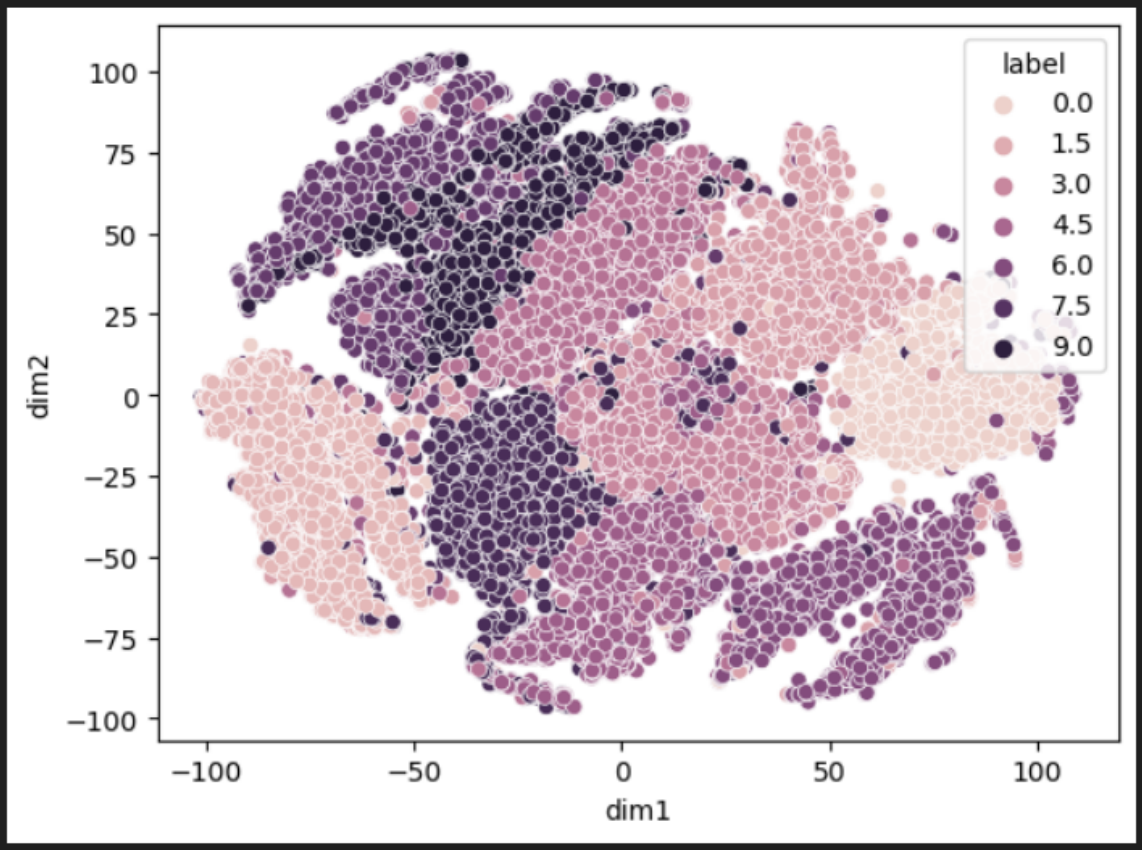
✓ 0.0s

```
sns.set_palette("husl")
```

```
sns.scatterplot(tsne_df, x="dim1", y="dim2", hue="label")
```

✓ 2.7s

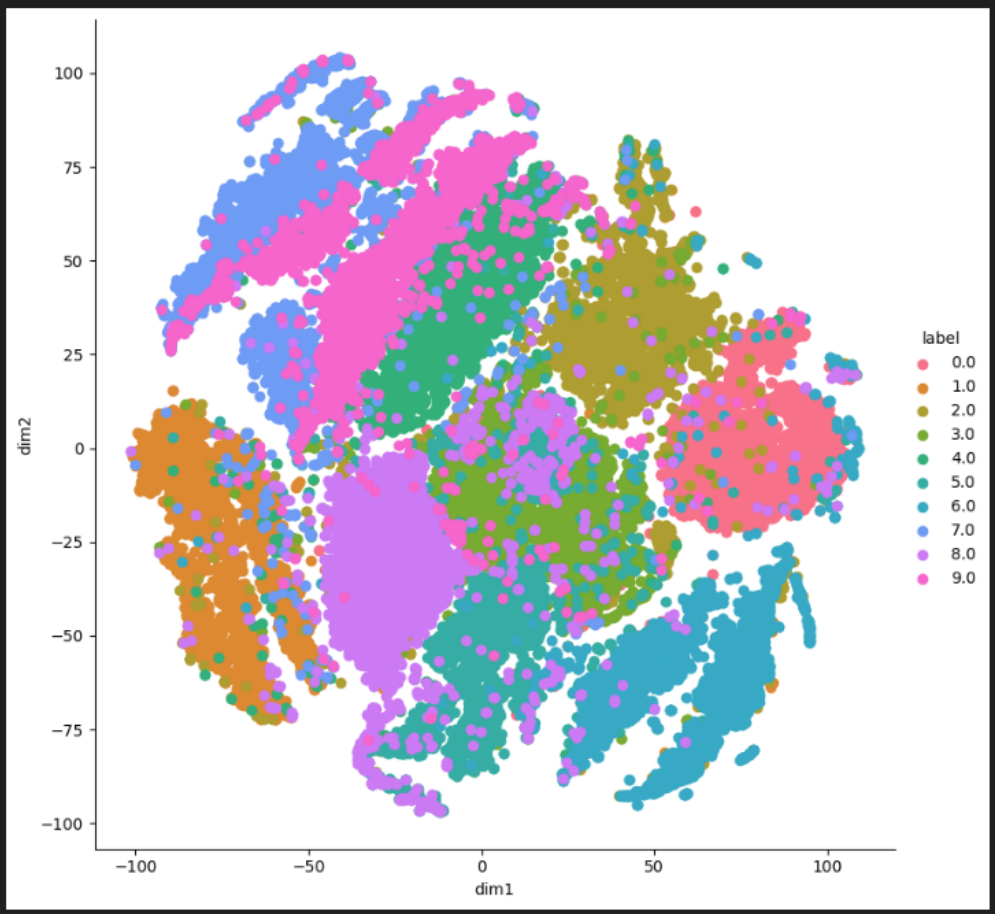
```
<AxesSubplot: xlabel='dim1', ylabel='dim2'>
```




```
sns.FacetGrid(tsne_df, hue="label", height=8).map(plt.scatter, "dim1", "dim2").add_legend()
```

✓ 0.7s

```
<seaborn.axisgrid.FacetGrid at 0x7f0f6156b010>
```

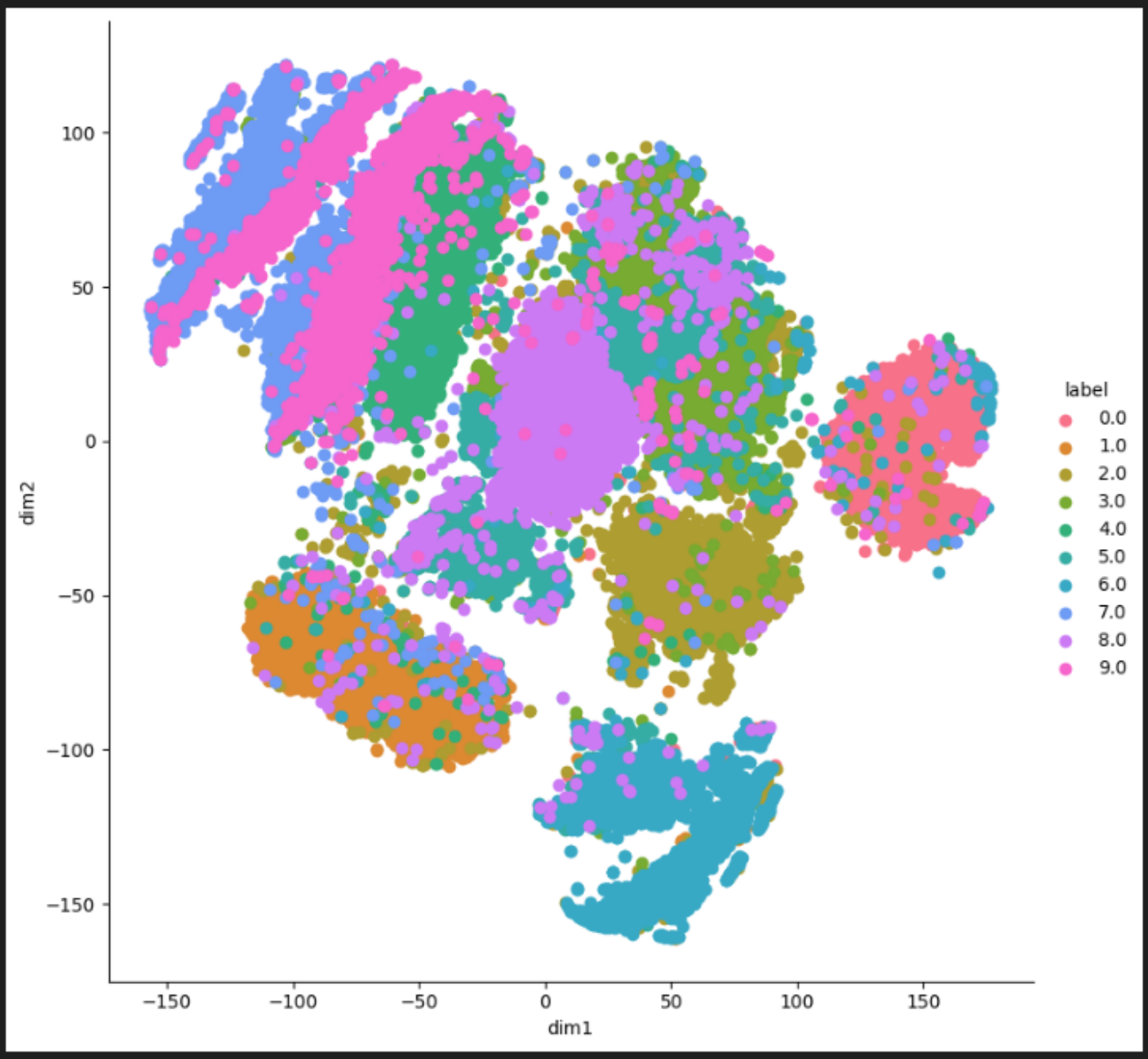


doing 10000 iterations, perplexity = 100

```
tsne = TSNE(n_components=2, random_state=10, perplexity=100, n_iter=10000)
sd1 = standardized_data
lb1 = labels
tsne_data1 = tsne.fit_transform(sd1)
tsne_data_l1 = np.vstack((tsne_data1.T, lb1)).T
tsne_df1 = pd.DataFrame(tsne_data_l1, columns=["dim1", "dim2", "label"])
sns.FacetGrid(tsne_df1, hue="label", height=8).map(plt.scatter, "dim1", "dim2").add_legend()
```

4] ✓ 28m 40.1s

```
<seaborn.axisgrid.FacetGrid at 0x7f0f61339190>
```




took fucking 28 minutes